

请用C语言+寄存器的方式改写以下程序：

例6.1和例6.2

(注意NVIC可以用固件库函数写)

例6.1

```
void TIM_Configuration(void)
{
    RCC->APB1ENR |= (1 << 0) | (1 << 1);
    RCC->APB2ENR |= 1 << 11;
    TIM1->RCR = 4;
    TIM1->ARR = 10 * 1000 - 1;
    TIM1->PSC = SystemCoreClock / 10000 - 1;
    TIM2->CR1 &= ~(1 << 4);
    TIM2->DIER |= 1 << 0;
    TIM2->CR1 |= 1 << 0;
    TIM3->ARR = 10 * 500 - 1;
    TIM3->PSC = SystemCoreClock / 10000 - 1;
    TIM3->CR1 &= ~(1 << 4);
    TIM3->DIER |= 1 << 0;
    TIM3->CR1 |= 1 << 0;
}
```

```
void TIM1_UP_IRQHandler(void)
{
    TIM1->SR &= ~(1 << 0);
    TIFlag = 1;
}
```

请用C语言+寄存器的方式改写以下程序：

例6.1和例6.2

(注意NVIC可以用固件库函数写)

例6.1续

```
void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1 << 0) == 1)
    {
        TIM2->SR &= ~(1 << 0);
        T2Flag = 1;
    }
}
```

```
void TIM3_IRQHandler(void)
{
    if (TIM3->SR & (1 << 0) == 1)
    {
        TIM3->SR &= ~(1 << 0);
        T3Flag = 1;
    }
}
```

请用C语言+寄存器的方式改写以下程序：

例6.1和例6.2

(注意NVIC可以用固件库函数写)

例6.2

```
void TIM_Configuration(void)
{
    RCC->APB1ENR |= (1 << 1);
    TIM3->ARR = 65535;
    TIM3->PSC = SystemCoreClock / 2 / 3600 - 1;
    TIM3->CR1 &= ~(1 << 4);
    TIM3->DIER |= (1 << 1) | (1 << 2) | (1 << 3) | (1 << 4);
    TIM3->CCR1 = 72000 / 2;
    TIM3->CCR2 = 72000 / 4;
    TIM3->CCR3 = 72000 / 8;
    TIM3->CCR4 = 72000 / 16;
    TIM3->DIER &= ~(1 << 0);
    TIM3->CR1 |= 1 << 0;
}
```

```
void TIM3_IRQHandler(void)
{
    if (TIM3->SR & (1 << 1) == 1)
    {
        TIM3->SR &= ~(1 << 1);
        GPIOD->ODR ^= 1 << 2;
        TIM3->CCR1 += 72000 / 2;
    }
}
```

请用C语言+寄存器的方式改写以下程序：

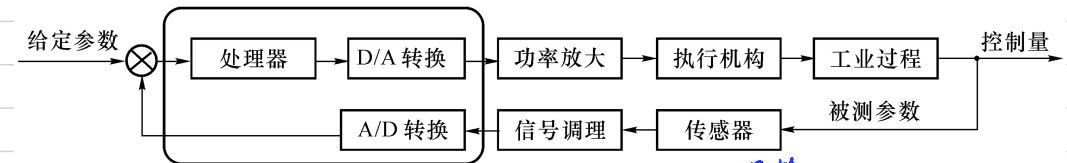
例6.1和例6.2

(注意NVIC可以用固件库函数写)

例6.2续

```
else if (TIM3->SR & (1 << 2) == 1)
{
    TIM3->SR &= ~(1 << 2);
    GPIOD->ODR ^= 1 << 3;
    TIM3->CCR2 += 72000 / 4;
}
else if (TIM3->SR & (1 << 3) == 1)
{
    TIM3->SR &= ~(1 << 3);
    GPIOD->ODR ^= 1 << 4;
    TIM3->CCR2 += 72000 / 8;
}
else if (TIM3->SR & (1 << 4) == 1)
{
    TIM3->SR &= ~(1 << 4);
    GPIOD->ODR ^= 1 << 7;
    TIM3->CCR2 += 72000 / 16;
}
}
```

7-1 基于嵌入式处理器的模拟输入输出系统是怎么构成的？各部分的主要功能是什么？
系统构成如下图所示。



传感器：感知或采集工业过程中的相关参数，如温度、压力、流量等。

信号调理电路：对传感器送来的信号进行调整处理，使采集的信号在A/D的有效范围内。

A/D转换器：将模拟信号转换为数字信号。

处理器：根据系统需求对转换后的数字信号进行数字处理或运算。

D/A转换器：将数字信号转换为模拟信号。

功率放大：对模拟信号进行功率放大以推动后续的执行机构。

执行机构：将电能转换为机械能或其他形式的能量以控制工业过程。

7-12 对于STM32F10x微控制器，使用片上ADC通道3(PA3作为ADCIN3)作为电位器位置检测的一个实例，如图7.41所示。当旋转电位器VR1时，ADCIN3随之电压发生变化，如果电位代表位置，假设0V表示位置0米，3.3V表示100米，当电位器从最低端向最高端旋转时，电位从0~3.3V变化，也即位置从0~100米线性变化。试回答：

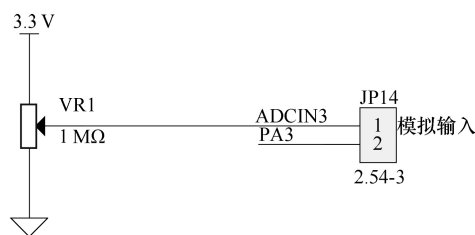


图 7.41 片上 DAC 应用

(1) 写出对片上 DAC 的初始化程序片段(含引脚配置及 DAC 初始化)。

(2) 写出采集电位并进行标度变换的程序。

(1)

```
void ADC_Configuration(void)
{
```

```
    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    ADC_InitTypeDef ADC_InitStructure;
```

```
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | \
                             RCC_APB2Periph_AFIO | \
                             RCC_APB2Periph_ADC1, ENABLE);
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
```

```
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
```

```
    GPIO_Init(ADC1, &GPIO_InitStructure);
```

7-12 对于STM32F10x 微控制器,使用片上 ADC 通道 3(PA3 作为 ADCIN3)作为电位器位置检测的一个实例,如图 7.41 所示。当旋转电位器 VR1 时,ADCIN3 随之电压发生变化,如果电位代表位置,假设 0 V 表示位置 0 米,3.3 V 表示 100 米,当电位器从最低端向最高端旋转时,电位从 0~3.3 V 变化,也即位置从 0~100 米线性变化。试回答:

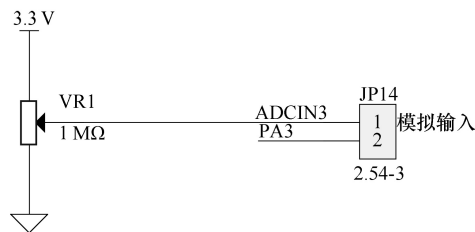


图 7.41 片上 DAC 应用

(1) 写出对片上 DAC 的初始化程序片段(含引脚配置及 DAC 初始化)。

(2) 写出采集电位并进行标度变换的程序。

(1) 续

```
ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
ADC_InitStructure.ADC_ScanConvMode = ENABLE;
ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
ADC_InitStructure.ADC_ExternalTrigConv = \
    ADC_ExternalTrigConv_None;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfChannel = 1;
ADC_Init(ADC1, &ADC_InitStructure);
ADC-RegularChannelConfig(ADC1, ADC_Channel_7, 1, \
    ADC_SampleTime_239Cycles5);
ADC_Cmd(ADC1, ENABLE);
ADC-ResetCalibration(ADC1);
while (ADC_GetResetCalibrationStatus(ADC1));
ADC-StartCalibration(ADC1);
while (ADC_GetCalibrationStatus(ADC1));
ADC-SoftwareStartConvCmd(ADC1, ENABLE);
}
```

(2)

```
uint16_t Temp;
float Voltage;
while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
Temp = ADC_GetConversionValue(ADC1);
Voltage = 3300 * Temp / 0xFFF;
```

7-14 简述利用片内 DAC 再通过外加运算放大电路产生指定幅度和周期的正弦波的方法和步骤。写出对 STM32F10x 的 DAC 进行配置和初始化的程序片段。

假设要输出振幅为A, 周期为T的正弦波, 首先采集N个点的正弦值, 其最大值要与A相等, 并将其存入内存缓冲区中, 然后控制两点之间输出间隔为 $T / (N - 1)$, 即可得到要求的正弦波。

```
void DAC_Configuration(void)
{
    GPIO_InitTypeDef  GPIO_InitStructure;
    DAC_InitTypeDef    DAC_InitStructure;
    RCC_APB1PeriphClockCmd(RCC_APB2Periph_DAC, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | \
                            RCC_APB2Periph_AFIO, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
    DAC_DeInit();
    DAC_InitStructure.DAC_Trigger = DAC_Mode_T3_TRGO;
    DAC_InitStructure.DAC_WaveGeneration = \
        DAC_WaveGeneration_None;
    DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Disable;
    DAC_Init(DAC_Channel_1, &DAC_InitStructure);
    DAC_Cmd(DAC_Channel_1, ENABLE);
    DAC_SetChannel1Data(DAC_Align_12b_R, 0);
}
```

7-15 试设计一个模拟输入输出系统,采用 STM32F10x 微控制器,采用 3.3 V 供电,内置 12 位 ADC,某压力传感器输出 0~100 mV 对于 0~10 MPa,压力超过 8.5 MPa 时,输出报警信号,让蜂鸣器发声,低于 8 MPa 时解除报警,并将得到的压力用 4~20 mA 电流输出到外部。写出相关程序片段。

```
uint16_t ScalingTransformation(void)
{
    uint16_t ADC_Data;
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);
    while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
    ADC_Data = ADC_GetConversionValue(ADC1);
    return (uint16_t) 0.1 * ADC_Data;
}

main(void)
{
    uint16_t Y_x = ScalingTransformation();
    if (Y_x > 8.5)
        Beep();
    else if (Y_x < 8)
    {
        ~Beep();
        uint16_t value = 124 + (Y_x - 4) * \
                        (620 - 124) / (8 - 4);
        DMA_SetChannel1Data(DAC_Align_12b_R, value);
    }
}
```